

WEEK 3 LEARNING GUIDE

BUILDING A DATA-DRIVEN SERVERLESS FEEDBACK API

1. INTRODUCTION

After building a basic serverless feedback application in the previous week, the focus of Week 3 shifted towards making the application more scalable, secure, and closer to a production-oriented system. While the earlier implementation successfully demonstrated how AWS Lambda, API Gateway, Amazon S3, and Cognito could be used together to build a serverless application, it also exposed several limitations. Storing feedback as individual JSON files in Amazon S3 worked well for learning purposes, but it was not an ideal solution for managing structured application data as the application grew.

The primary objective of this week was to redesign the backend architecture by introducing Amazon DynamoDB as the primary data store while continuing to use Amazon S3 for storing file attachments. This separation of structured metadata from binary file storage is a common design pattern in serverless applications and significantly improves scalability and maintainability.

Another major focus was understanding how application requirements influence database design. Unlike relational databases where tables are often designed around entities and relationships, DynamoDB requires developers to think in terms of access patterns. This represented a different way of approaching database design and became one of the most important concepts learned during the week.

Apart from redesigning the data storage layer, the application was also extended with features expected from a real feedback management system:

- Authenticated and anonymous feedback submission.
- Role-Based Access Control (RBAC) using Amazon Cognito.
- CRUD operations for user-owned feedback.
- Administrative access to view feedback from all users.
- Support for optional file attachments.
- A transactional upload workflow using Amazon S3 presigned URLs.

Rather than implementing these features independently, the goal was to understand how they interact within a complete serverless architecture. Throughout the implementation, emphasis was placed on keeping the frontend lightweight while ensuring that all important business logic, authorization, and validation remained the responsibility of the backend.

This week also involved considerably more architectural planning than the previous weeks. Before implementing new APIs, significant time was spent discussing the database schema, API design, authorization rules, and attachment workflow. Although this required more upfront effort, it reduced unnecessary refactoring later and resulted in a cleaner overall architecture.

By the end of the week, the application had evolved from a simple feedback collection API into a multi-user serverless feedback management system with a much stronger focus on scalability, security, and maintainability. More importantly, I developed a better understanding of how individual AWS services work together to build a complete cloud-native application rather than viewing them as isolated services.

2. CORE CONCEPTS LEARNED

AMAZON DYNAMODB

One of the biggest additions this week was **Amazon DynamoDB**, AWS's fully managed NoSQL database service. Unlike the previous implementation where feedback was stored as JSON files in Amazon S3, DynamoDB is designed specifically for storing and retrieving structured application data with low latency at any scale.

Before implementing DynamoDB, I spent time understanding how NoSQL databases differ from traditional relational databases. The biggest shift in thinking was realizing that DynamoDB tables are designed around **how the application accesses data**, rather than how the data is related.

What I understood

- DynamoDB stores data as **Items** inside **Tables**.
- Each item is a collection of **Attributes**, similar to a JSON object.
- Unlike relational databases, there are no joins or foreign keys.
- Tables should be designed around the application's **access patterns** instead of normalization.

Application in this project

DynamoDB replaced Amazon S3 as the application's primary data store.

Instead of storing every feedback as a JSON file:

feedback1.json

feedback2.json

feedback3.json

feedback became structured database records.

Feedback Table

ownerId

feedbackId

title

content

createdAt

lastUpdated

attachments

This change made CRUD operations significantly simpler while preparing the application for larger datasets.

Example item:

```
{  
  "ownerId": "user123",  
  "feedbackId": "fb001",  
  "title": "Feedback Title",  
  "content": "Sample feedback"  
}
```

Primary Keys

One of the most important topics covered this week was DynamoDB key design.

Unlike SQL databases where primary keys mainly ensure uniqueness, DynamoDB keys directly influence how data is stored and retrieved.

The final table used a composite primary key.

Partition Key : ownerId

Sort Key : feedbackId

This allowed:

- Multiple feedback entries for a single user.
- Efficient retrieval of all feedback belonging to one user.
- Unique identification of every feedback item.

Instead of storing unrelated records together, each user's feedback naturally became grouped under the same partition.

Designing Around Access Patterns

One concept that appeared repeatedly throughout the implementation was **Access Pattern Design**.

Rather than asking,

"What information do I want to store?"

the design process became,

"What questions will my application ask?"

For the feedback application, the major access patterns were:

- Get all feedback submitted by a user.
- Get a specific feedback.
- Create new feedback.
- Update existing feedback.
- Delete feedback.
- Allow administrators to view all feedback.

Once these access patterns were identified, the DynamoDB schema became much easier to design.

This approach was very different from relational database design and was one of the biggest conceptual takeaways of the week.

DynamoDB Data Types

Although DynamoDB is schemaless, every attribute still has an associated data type.

The feedback application primarily used:

- **String** – titles, feedback content, timestamps, IDs.
- **List** – attachment metadata.
- **Boolean** – flags where required.

Compared to relational databases, DynamoDB allows different items within the same table to contain different attributes. This flexibility made it easier to extend the feedback model without requiring schema migrations.

Secondary Indexes

The primary key supported user-based queries, but administrators also needed to retrieve feedback across all users.

Instead of scanning the entire table, DynamoDB indexes were introduced.

The final implementation included:

- **Local Secondary Index (LSI)** using **lastUpdated**
- **Global Secondary Index (GSI)** using **entityType** and **lastUpdated**

Simplified representation:

Main Table

PK : ownerId

SK : feedbackId

↓

GSI

PK : entityType

SK : lastUpdated

This allowed administrators to efficiently retrieve all feedback without affecting the primary access pattern used by regular users.

The biggest lesson here was that indexes in DynamoDB are **not optional optimizations**. They are often designed alongside the table itself because they support additional access patterns that the primary key cannot satisfy.

Query vs Scan

One of the most important DynamoDB concepts I learned was the difference between **Query** and **Scan** operations.

Although both can retrieve data from a table, they behave very differently.

Query

Uses Partition Key



Reads Matching Items

Scan

Reads Entire Table



Filters Matching Items

What I understood

- **Query** retrieves items using the table's partition key or an index.
- **Scan** examines every item in the table before returning matching results.
- Query operations are significantly faster and more cost-effective than scans.
- DynamoDB tables should be designed so that most application requests use queries rather than scans.

Application in this project

The feedback API was designed so that almost every database operation could use a **Query** instead of a **Scan**. Regular users retrieved feedback using the **ownerId** partition key, while administrators queried the Global Secondary Index to retrieve all feedback efficiently. Designing the schema around these access patterns avoided expensive table scans and demonstrated why database design should be driven by how the application retrieves data rather than simply how the data is stored.

BOTO3 WITH DYNAMODB

After designing the database schema, the next step was interacting with DynamoDB programmatically using **Boto3**.

Since the previous week already introduced Boto3 while working with Amazon S3, this week focused on using it with DynamoDB resources.

The DynamoDB table was initialized as:

```
table = dynamodb.Table(TABLE_NAME)
```

Some of the operations used throughout the project included:

```
table.put_item(...)
```

Creates a new feedback entry.

```
table.get_item(...)
```

Retrieves a single feedback.

```
table.update_item(...)
```

Updates an existing feedback.

```
table.delete_item(...)
```

Deletes a feedback record.

```
table.query(...)
```

Retrieves feedback belonging to a specific user.

This became the preferred method throughout the project because it efficiently retrieves items using the partition key instead of scanning the table.

What I learned

Compared to Amazon S3, interacting with DynamoDB felt much closer to working with application data.

Instead of:

- Reading files
- Parsing JSON
- Combining responses

the application could directly retrieve the required items using the appropriate keys.

This greatly simplified the Lambda implementation while improving performance.

AMAZON S3 PRESIGNED URLS

Supporting file attachments introduced another important AWS concept: **Presigned URLs**.

Instead of uploading files through the backend, the application generates temporary URLs that allow clients to upload files directly to Amazon S3.

Simplified workflow:

Client

| Request Upload URL



AWS Lambda

| Generate Presigned URL



Amazon S3



Client uploads directly

What I understood

- Presigned URLs grant temporary permission to perform a specific operation.
- The S3 bucket remains private throughout the process.
- AWS credentials are never exposed to the client.
- Uploading directly from the browser avoids routing large files through Lambda, reducing execution time and improving scalability.

Application in this project

Each feedback submission began by requesting presigned upload URLs from the backend. Once all attachments were successfully uploaded, the frontend completed the submission by sending only the attachment metadata to the backend for storage in DynamoDB.

This workflow demonstrated how multiple AWS services can cooperate to securely handle large file uploads without routing file data through the backend.

ROLE-BASED ACCESS CONTROL (RBAC)

Another important concept introduced this week was **authorization**.

Although Cognito authentication had already been introduced previously, this week focused on determining **what authenticated users are allowed to do**.

The application supported three different user types.

Authenticated Users

Allowed to:

- Submit feedback.
- View their own feedback.
- Edit their own feedback.
- Delete their own feedback.

Administrators

Allowed to:

- View feedback from all users.
- Download all submitted feedback.
- View user details associated with each submission.

To prevent accidental modification of another user's data, administrators were intentionally restricted from editing or deleting feedback they did not own.

This decision kept ownership with the original author while still allowing administrators to review submissions.

Anonymous Users

Allowed to:

- Submit anonymous feedback.

Not allowed to:

- View previous submissions.
- Edit feedback.
- Delete feedback.

Since anonymous users have no identity associated with their submission, ownership-based operations would not be possible.

Application in this project

Authorization checks were performed after the user's identity had been verified by the API Gateway JWT Authorizer.

The Lambda function extracted the relevant user information from the JWT claims before determining which operations were permitted.

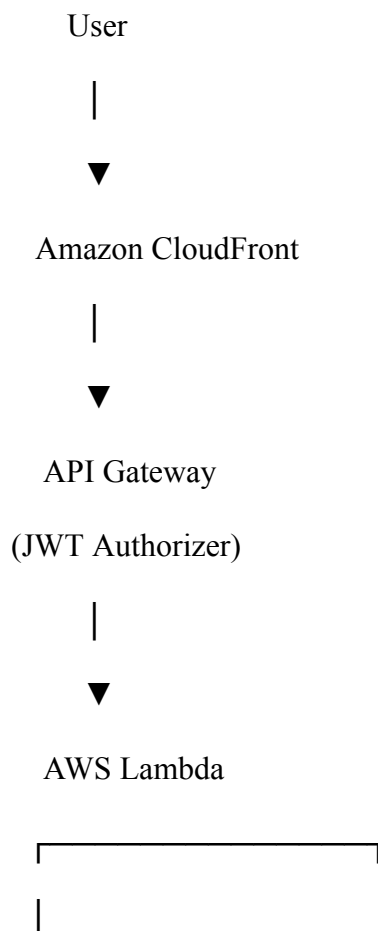
This kept authorization centralized in the backend instead of relying on frontend restrictions.

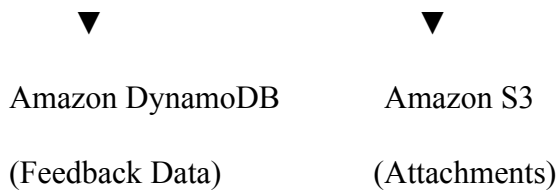
3. PROJECT IMPLEMENTATION

FINAL ARCHITECTURE

The biggest architectural change this week was separating **structured application data** from **binary file storage**.

Instead of storing everything inside Amazon S3, each service was given a dedicated responsibility.





This architecture followed a much cleaner separation of concerns:

- **DynamoDB** stores structured feedback information.
- **Amazon S3** stores uploaded attachment files.
- **Lambda** contains the business logic.
- **API Gateway** exposes the REST API.
- **Cognito** handles user authentication.
- **CloudFront** continues serving the frontend securely.

Compared to the previous week's implementation, the application became significantly more modular, with each AWS service performing a clearly defined responsibility.

DATABASE MIGRATION

One of the biggest learning outcomes this week was understanding why choosing the correct storage service is just as important as designing the application itself. Although Amazon S3 worked well during the previous week, introducing features such as editing feedback, ownership checks, administrator queries, and attachment metadata made it clear that object storage was no longer the right solution for structured application data.

Previously, every feedback submission was stored as an individual JSON file.

feedback-storage/

├─ 1.json

├─ 2.json

├─ 3.json

Although this approach worked well for learning the basics of AWS, it became increasingly inefficient as more application features were introduced.

Some limitations observed were:

- Retrieving all feedback required reading every object individually.
- Updating a single feedback involved rewriting the complete JSON object.
- Searching or filtering feedback was inefficient.
- There was no straightforward way to query data based on ownership or timestamps.

Migrating to DynamoDB solved these limitations by storing each feedback as an independent database item.

```
{  
  
  "ownerId": "user123",  
  
  "feedbackId": "fb001",  
  
  "title": "Application Feedback",  
  
  "content": "The UI is intuitive.",  
  
  "createdAt": "...",  
  
  "lastUpdated": "...",  
  
  "attachments": []  
  
}
```

This migration greatly simplified CRUD operations while also preparing the backend for future scalability.

DESIGNING THE FEEDBACK SCHEMA

Unlike previous projects where the database schema was largely predefined, this week required designing the DynamoDB table from scratch.

The final schema was driven by the application's access patterns rather than simply listing all available fields.

The primary key consisted of:

Partition Key : ownerId

Sort Key : feedbackId

This design allowed:

- Multiple feedback entries per user.
- Efficient retrieval of a user's feedback.
- Unique identification of every submission.

Additional attributes included:

- title
- content
- createdAt
- lastUpdated
- attachments
- ownerType
- entityType

The inclusion of `entityType` initially seemed unnecessary because the table only stored feedback records. However, using a constant partition key for the Global Secondary Index allowed administrators to efficiently retrieve every feedback item without performing a full table scan.

This was one of the first examples where the database schema was designed specifically around query requirements rather than data organization.

TWO-PHASE FEEDBACK SUBMISSION

Supporting file attachments introduced an interesting design challenge.

Initially, it seemed reasonable to create the feedback record first and upload the attachments afterwards.

However, this approach could leave incomplete feedback records if the upload failed midway.

To avoid this problem, the submission process was divided into two stages.

Step 1

Client

|



POST /feedback/init

|



Generate Feedback ID

Generate Presigned URLs

|



Return Upload URLs

Step 2

Client uploads files

|



Amazon S3

Step 3

POST /feedback/complete

|



Verify uploads

|



Store metadata in DynamoDB

This workflow ensured that the database was only updated after all attachments had been uploaded successfully.

Although it required two API calls instead of one, the implementation became much more reliable and prevented incomplete records from being stored.

This was one of the most valuable architectural lessons of the week, as it demonstrated how breaking a process into multiple stages can improve data consistency.

ATTACHMENT VERIFICATION

Generating presigned URLs alone was not sufficient.

A user could receive upload URLs but never actually upload the files.

If the backend immediately stored attachment metadata, the application would contain references to files that did not exist.

To prevent this, the **complete** endpoint first verified that every attachment had been uploaded successfully.

Simplified workflow:

Client

|

Uploads Files

|

Amazon S3

|

Verify Objects Exist

|

Yes —————> Save Feedback

No

|

Delete Uploaded Files

|

Return Error

If verification failed:

- Any partially uploaded files were removed.
- No feedback record was written to DynamoDB.
- The client received an appropriate error response.

Although this added additional backend logic, it ensured that the database and storage bucket always remained consistent.

One important architectural decision was keeping uploaded files and application metadata in separate services. Amazon S3 stores binary attachment files efficiently, while DynamoDB stores structured information such as feedback content, timestamps, ownership details, and attachment references. Separating these responsibilities allowed each AWS service to be used for the purpose it was designed for instead of forcing a single service to handle both structured and unstructured data.

DESIGNING REST APIS

As the application grew, exposing every operation through a single endpoint was no longer practical. Different operations such as creating feedback, uploading attachments, retrieving submissions, and managing anonymous feedback each required different request flows. Designing a clear REST API helped organize these operations while keeping the backend easier to maintain and extend.

The final API included endpoints for both authenticated and anonymous users.

Method	Endpoint	Purpose
POST	<code>/feedback/init</code>	Generate feedback ID and presigned upload URLs
POST	<code>/feedback/complete</code>	Verify uploads and store feedback
POST	<code>/feedback/anonymous/init</code>	Anonymous upload initialization
POST	<code>/feedback/anonymous/complete</code>	Complete anonymous feedback submission
GET	<code>/feedback</code>	Retrieve authenticated user's feedback
GET	<code>/feedback/admin</code>	Retrieve all feedback for administrators
GET	<code>/feedback/{ownerId}/{feedbackId}</code>	Retrieve a single feedback
GET	<code>/feedback/{ownerId}/{feedbackId}/download</code>	Download feedback and attachments
PUT	<code>/feedback/{ownerId}/{feedbackId}</code>	Update feedback
DELETE	<code>/feedback/{ownerId}/{feedbackId}</code>	Delete feedback

Rather than implementing separate Lambda functions for every endpoint, all routes were handled by a single Lambda function.

```
def lambda_handler(event, context):  
  
    method = event["requestContext"]["http"]["method"]  
    path = event["rawPath"]  
  
    ...
```

The handler determined both the HTTP method and request path before routing the request to the appropriate helper function.

This approach reduced deployment complexity while keeping related business logic together.

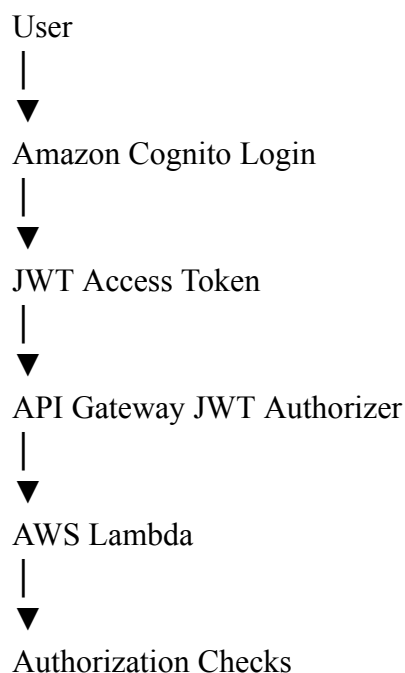
AUTHENTICATION AND AUTHORIZATION

Authentication was handled by **Amazon Cognito**, while authorization logic remained inside the Lambda function.

When a request reached API Gateway:

1. The JWT token was validated.
2. User information was extracted from the token.
3. Lambda determined whether the user was an administrator or a regular user.
4. The requested operation was validated before processing.

Simplified workflow:



This separation ensured that authentication and business logic remained independent.

API Gateway verified **who** the user was.

Lambda decided **what the user was allowed to do**.

OWNERSHIP-BASED ACCESS CONTROL

A key design decision was ensuring that users could only modify feedback they owned.

Before performing an update or delete operation, Lambda compared the feedback owner with the authenticated user.

Simplified logic:

```
if owner_id != current_user and not is_admin:  
    return unauthorized()
```

This prevented users from modifying another user's feedback simply by changing the URL.

Administrators were intentionally limited to viewing all feedback rather than editing or deleting submissions created by other users.

This decision helped preserve ownership while still giving administrators the visibility they required.

SUPPORTING ANONYMOUS FEEDBACK

The application also supported anonymous submissions.

Unlike authenticated feedback, anonymous submissions were not linked to a Cognito user.

Instead, the backend stored a different owner type while preventing future retrieval or modification.

This resulted in two independent submission workflows.

Authenticated users:

Login

|



Submit Feedback



View / Edit / Delete

Anonymous users:

Submit Feedback



Store Feedback



No Further Access

Although both workflows ultimately stored data in DynamoDB, the authorization rules remained completely different.

4. KEY LEARNINGS & TAKEAWAYS

Week 3 marked a significant shift from learning individual AWS services to designing complete serverless systems.

Some of the most important takeaways from this week are:

- DynamoDB tables should be designed around application access patterns rather than traditional relational database principles.
- Structured application data and binary file storage should be handled by different AWS services based on their strengths.
- Authentication and authorization solve different problems and should remain separate responsibilities.
- Backend services should always enforce authorization rules instead of relying on frontend validation.
- Breaking large workflows into smaller stages improves reliability and makes failures easier to handle.
- Designing backend systems requires considering database structure, API design, authorization, and storage together rather than as independent components.
- Building a scalable backend requires thinking beyond functionality and considering data consistency, security, and maintainability from the beginning.

Compared to the previous weeks, much more time was spent planning the architecture before implementation. Although this initially slowed down development, it resulted in a cleaner design and significantly reduced the amount of rework required later.

Overall, Week 3 marked a transition from learning individual AWS services to understanding how complete cloud-native systems are designed. Rather than focusing on isolated features, the implementation required every architectural decision to consider scalability, security, data consistency, and maintainability together.